5-5 · 파티셔닝과 셔플링 최적화 · 정리·회고 0.5H

정리·회고 및 산출물 점검

나눠서 하되, 나누는 비용을 안다 — 5주차 마무리

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① 5주차 닷새의 배운 것과 남은 결과물을 한 표로 되짚는다

② 체크리스트와 튜닝 표로 어디부터 볼지 안다

③ 자주 겪는 오류 여섯 가지의 원인과 해결을 안다

④ 산출물(mart.py + 마트 조회 결과) 마감 10-30(금) 23:59

끝은 stop 까지 — down -v 는 MinIO 볼륨까지 지워 이번 주 테이블이 사라집니다

오늘의 요약

5주차에 지나온 길

차시	배운 것	남은 결과물
5-1	분산 처리 · 지연 평가 · Spark UI	UI 를 읽는 눈
5-2	마스터-워커 클러스터와 잡 제출	docker-compose.yml
5-3	s3a · Iceberg 연결	lake.db.orders
5-4	DataFrame API 와 SparkSQL	product_sales · daily_revenue
5-5	파티셔닝 · 셔플 최적화	mart.py · daily_sales_mart

한 주를 관통한 생각 — 「나눠서 하되, 나누는 비용을 안다」 병렬은 공짜가 아니고 셔플이 그 값이다

튜닝 체크리스트

증상 → 먼저 볼 곳 → 흔한 처방

증상	먼저 볼 곳	흔한 처방
전체가 느리다	Stage 별 시간	가장 긴 Stage 의 Exchange
한 태스크만 오래	Task 시간 Max/Median	스큐 — 키 분포 확인
메모리 부족	파티션 수	repartition 으로 조각을 작게
작은 파일이 많다	저장 직전 파티션 수	coalesce 로 정리
조인이 느리다	계획의 조인 방식	작은 쪽 브로드캐스트
빈 태스크가 많다	shuffle.partitions	데이터 크기에 맞게 줄이기

측정 없이 정하지 않는다 — 처음엔 기본값으로 돌리고 UI 를 본다

오늘의 체크리스트

15개 중 핵심 여덟 개

1. 파티션이 태스크 병렬성의 단위임을 설명한다
2. 좁은·넓은 변환을 구분하고 셔플이 생기는 지점을 짚는다
3. repartition(셔플 있음)과 coalesce(셔플 없음) 차이를 확인했다
4. groupBy 전후 Stage 와 Shuffle Read/Write 를 UI 에서 봤다
5. 실행 계획에서 Exchange 를 찾아 셔플 여부를 판정했다
6. 스큐를 만들어 한 태스크만 오래 걸리는 것을 확인했다
7. mart.py 를 spark-submit 으로 실행해 마트 테이블을 만들었다
8. 인자·로그·종료 코드를 넣어 실무형으로 다듬었다

나머지 일곱 개(분포 재기·멱등·제출 정리 등)는 사이트 본문에서 확인해 보세요

자주 겪는 오류 1/2

잡이 돌지 않을 때

증상	원인	해결
Initial job has not accepted any resources	열어 둔 PySpark 셸이 워커 코어 4개를 잡음	8080 에서 셸 확인 → 셸이면 exit()
localhost:4040 이 열리지 않는다	셸·잡이 떠 있을 때만 열림 override 없으면 컨테이너 안	s0 3단계 override 파일 → up -d → 셸 띄운 뒤
집계 중 익스큐터가 OutOfMemoryError	파티션이 너무 적거나 한 키에 몰림(스큐)	repartition 으로 잘게 키 분포는 groupBy 로

첫 줄은 15초마다 찍히는 WARN — 셋 다 오늘 실습에서 실제로 만난 증상

자주 겪는 오류 2/2

파일 수와 coalesce

증상	원인	해결
MinIO 에 자잘한 Parquet 이 여러 개	저장 직전 파티션 수 = 출력 파일 수(test-narrow 4)	저장 전 coalesce(1) 또는 repartition(n) 으로 정리
coalesce(16) 인데 파티션 수 그대로	coalesce 는 합치기만 늘릴 수 없다	늘릴 땐 repartition(16) 전체 셔플이니 필요한지 먼저
큰 결과에 coalesce(1) 잡이 죽는다	한 태스크가 전체를 떠안는다	repartition(n) 으로 적당히 파일당 100~200MB 기준

AQE 를 끄면 셔플 파티션 200개만큼까지 파일이 생길 수 있다

예상 질문

자주 나오는 질문 네 가지

1. **shuffle.partitions 를 항상 줄이면 좋나요?**
   아니다 — 데이터가 크면 늘려야 한다. AQE 가 켜져 있으면 대개 알아서 조정하니 문제가 실제로 보일 때만

2. **셔플을 아예 없앨 수는 없나요?**
   groupBy·join 이 있으면 원리상 필요 — 필터 먼저·열 줄이기·브로드캐스트로 줄이는 것이 현실적 목표

3. **cache 와 repartition 중 뭘 먼저 하나요?**
   여러 번 쓸 데이터면 cache, 병렬이 모자라면 repartition — 둘 다 필요하면 repartition 후 cache

4. **실무에서 파티션 수를 어떻게 정하나요?**
   기본값으로 돌리고 UI 를 본다 — Task 가 수십 ms 면 줄이고, 메모리가 부족하면 늘린다

컨테이너 정돈

수업 끝 — stop 하고 6주차 폴더

1. **cd ~/data-eng/week5 → ls**
   파일 여섯 — compose · override · first_job · lake-pyspark · mart.py · mart.log

2. **docker compose stop**
   셸(>>>)이 열려 있으면 exit() 로 먼저 나온 뒤 — 6주차 Kafka 는 메모리를 많이 쓴다

3. **docker compose ps -a**
   넷이 Exited 로 남으면 끝 — 컨테이너·볼륨은 그대로

4. **mkdir -p ~/data-eng/week6**
   다음 회차 작업 폴더를 미리 만들어 둔다

docker compose 는

stop 만

(-v 금지)

다음 회차 예고

6-1 · 실시간 데이터와 Kafka (10-30 금)

01

이론 1H

실시간 데이터의 특징

배치 vs 스트리밍

02

이론·실습 2.5H

Kafka 핵심 구조

CLI 로 첫 송수신

03

정리·회고 0.5H

체크리스트 점검

용어 복습 · 질의응답

준비: Docker Desktop 실행(메모리 8GB 이상 권장) · docker --version 확인 · 2주차 MySQL 컨테이너는 6-4 CDC 에서 다시 쓰니 지우지 말 것

5-5 · 정리·회고 0.5H 마무리

다음 회차는 6-1 실시간 데이터와 Kafka

5주차까지는 「모아 두었다가 한꺼번에」, 6주차는 「생기는 대로 흘려보내며」 — 5주차 산출물은 10-30(금) 23:59 까지

- ✓ 배치 vs 스트리밍 — 처리 단위·지연시간
- ✓ 브로커·토픽·파티션·오프셋
- ✓ Docker 로 KRaft 단일 브로커 기동
- ✓ CLI 로 토픽 만들고 메시지 송수신

준비할 것

Docker 8GB 이상 · Spark 는 stop